

Advanced Python for Data Science

Week(5): Web Development & API Design

Tuesday, 14th July 2026, 14:00-17:15

Dr. Ghadeer Mobasher

What We'll Cover Today



REST Architecture

The rules behind well-behaved web APIs



Building APIs with Flask

Routes, requests and JSON responses



Structure & HTTP Methods

GET, POST, PUT, PATCH, DELETE



Authentication (JWT)

Proving who you are, token by token



Deployment

From dev server to production WSGI



Docker

Packaging the app so it runs anywhere

1. REST ARCHITECTURE

A Restaurant, Not a Kitchen Tour



REpresentational State Transfer (REST) is a set of rules for how a client and server talk. Think of the **client** as a **diner** and the server as a restaurant kitchen: the diner never walks into the kitchen; they just place an order and get a plate back.

- **Stateless:** Each order ticket has everything the kitchen needs, no memory of past visits required.
- **Client-Server:** The dining room and kitchen can be redesigned independently.
- **Cacheable:** A menu says which dishes can be "reheated" (reused) safely.
- **Uniform Interface:** Every table orders the same way → GET, POST, PUT, DELETE.



The client never touches the server's internals → only the request/response "order ticket."

```
GET /users
→ returns all users

POST /users
→ creates a new user
```

Uniform interface: the same handful of verbs works for every resource in the API.

2. BUILDING APIs WITH FLASK

Flask = A Receptionist for Your App



Flask is a lightweight Python framework. You describe a URL (a "desk") and a function that handles whoever shows up asking for it.

```
from flask import Flask, request, jsonify
app = Flask("Adding New Users")
users = []

@app.route("/users", methods=["GET"])
def get_users():
    return jsonify(users)

@app.route("/users", methods=["POST"])
def add_user():
    data = request.get_json()
    users.append(data)
    return jsonify({"message": "User added"}), 201
```

How to read it?

users = [] an in-memory list standing in for a database.

@app.route(...) labels a URL, like putting a sign on a desk.

methods=["GET"] says which "request type" that desk accepts.

jsonify(...) wraps a Python dict/list into a JSON reply.

request.get_json() reads the JSON body a client sent in.

2. BUILDING APIs WITH FLASK

Calling the API like a Client



The requests library plays the role of a customer: it sends an order (POST) and later asks for the full list (GET).

```
import requests

url = "http://127.0.0.1:5000/users"
user = {"name": "Adam", "email": "adam@x.com"}

response = requests.post(url, json=user)
print(response.json())

all_users = requests.get(url).json()
print(all_users)
```

json=user auto-converts the dict to JSON and sets the right headers, you never touch a header manually.

Round trip

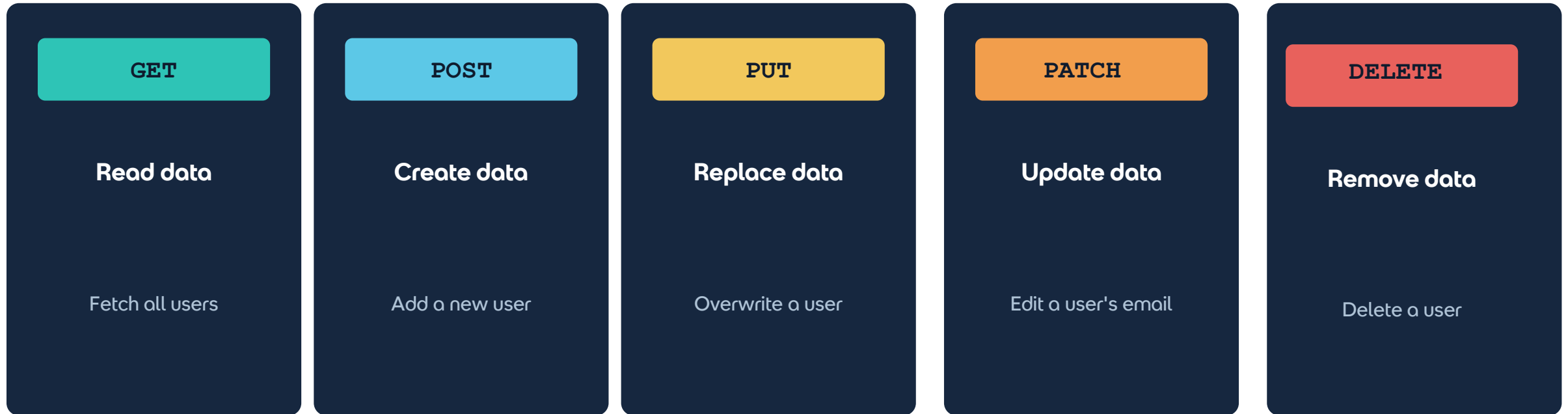
- 1** **POST /users { name, email }**
Client asks the server to add a user
- 2** **201 Created { message }**
Server confirms it was stored
- 3** **GET /users**
Client asks for the full list
- 4** **200 OK [users...]**
Server returns the current list

3. STRUCTURE & ROUTING



Five Verbs, One Filing Cabinet

Every HTTP method signals intent, for instance :



Recommended layout: routes/user_routes.py defines a Blueprint (self-contained); app.py registers every Blueprint centrally → like a directory that assigns each department its own hallway.

4. AUTHENTICATION (JWT)



A Wristband, Not a Guest List

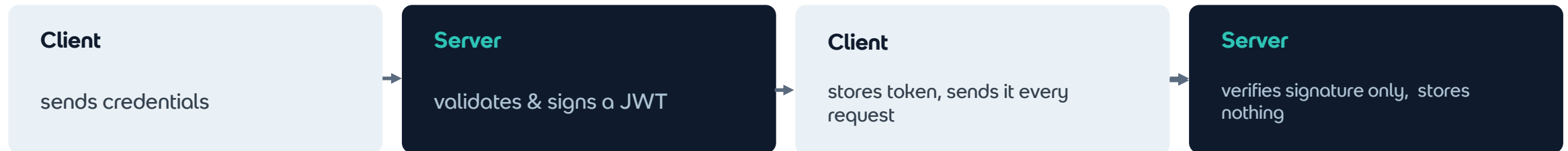
Token-based authentication works like a concert wristband: the server checks you once at the door, then a signed token proves who you are on every later visit.



Authentication verifies who you are.



Authorization verifies what you're allowed to do.



```
POST /login { username, password } → { access_token }
GET /protected Authorization: Bearer <token> → { "msg": "Access granted" }
```

Test Kitchen vs. Restaurant Service

`python app.py` is a dev server, great for tasting a single dish.

A WSGI server (Gunicorn/Waitress) is the production kitchen, built to handle a full dining room at once.



Local / Dev

```
python app.py
```

- Single-threaded, auto-reloads on save
- Fine for one developer testing locally
- Not built for real traffic or concurrency



Production

```
gunicorn production:app / waitress
```

- Gunicorn (Mac/Linux) or Waitress (Windows)
- Handles many concurrent requests
- Your Flask file only needs to expose app

6. DOCKER

A Shipping Container for your App



A shipping container holds the same cargo no matter which ship, truck, or port handles it. Docker does the same for code: package the app once, and it runs identically everywhere.

Dockerfile

the recipe → instructions to build the image

Docker Image

the blueprint → code + dependencies, frozen

Docker Container

a running instance of that image

Docker Hub

a shelf where images are stored & shared

Build → Run → Reach it

```
docker buildx build -t flask-api-dev \
    -f Dockerfile.dev .
```

```
docker run -p 5000:5000 flask-api-dev
```

```
→ curl http://localhost:5000/users
```

-p 5000:5000 maps your machine's port to the port inside the container → like plugging the container into the building's mail slot.

Key Takeaways

- REST is a contract: stateless requests, a uniform set of verbs, client and server evolving independently.
- Flask turns a URL + function into a route → `jsonify()` and `request.get_json()` move data in and out as JSON.
- HTTP methods signal intent: GET reads, POST creates, PUT replaces, PATCH edits, DELETE removes.
- JWT authentication checks you once, then trusts a signed token (the server stores nothing in between).
- A WSGI server (Gunicorn/Waitress) replaces the dev server for real, concurrent production traffic.
- Docker packages the app and its dependencies into one container that runs the same on any machine.

Pause / Break

See you in 15 minutes 😊!
Please be on time.

Team PySquad

15-20 minutes presentation

Team A

15-20 minutes presentation

Take-home Exercises

Solve Problems (1 -4) in "Week5_Exercise.ipynb". *Don't forget to split your screen. Read the whole instructions first and answer the reflection questions in your own words.*

Extending the Flask API You Built

- This notebook is a **companion to** `Week5.ipynb`. It does not repeat the theory but it gives you small, guided modifications to the files you already worked with.
- In addition to take-home mini challenge that pulls everything together (routing, JWT auth, and Docker).

Before you start

- Make sure you completed `Week5.ipynb` first. You'll be editing files you already opened there: `docker.py`, `server.py`, `client.py`, and `project/routes/user_routes.py`.
- Keep the **Week5 directory** open in your terminal/editor, same as before.
- Each exercise below tells you **which file to open**, **exactly what to add**, and **what output you should see** when it works. If your output doesn't match, that's the bug to fix. Don't move on until it does.

How this notebook is structured?

Exercise	File you edit	Concept practiced
1	<code>docker.py</code>	Adding a new route (DELETE)
2	<code>project/routes/user_routes.py</code>	Input validation on a Blueprint route
3	<code>server.py</code> + <code>client.py</code>	Reading identity out of a JWT
4	<code>docker.py</code> + <code>Dockerfile.dev</code>	Rebuilding a Docker image after a code change
Mini Challenge	new files you create	Combining routing + JWT + Docker

- The deadline is 19.07.2026 @ 23:59.
- See you next time when we dive into **Cloud Deployment & DevOps**

Bonus Mini Challenge



Mini Challenge: Secured, Dockerized "Delete User" API

This challenge combines all three ideas from this notebook: **routing** (Ex. 1), **JWT authentication** (Ex. 3), and **Docker** (Ex. 4). Nothing here is copy-paste → you're assembling pieces you've already built separately.

The goal

Build an API where:

- Anyone can POST `/login` with `{"username": "student", "password": "pass123"}` to get a JWT (reuse the logic from `server.py`).
- Anyone can GET `/users` **without** a token (public).
- Only someone with a valid JWT can DELETE `/users/<id>` (protected).
- The whole thing runs in a Docker container.

Step-by-step instructions

1. Create a new file `challenge_api.py` in your Week5 directory. Here is a skeleton → the **TODOs** are for you to fill in using what you did in the exercises above:

```
from flask import Flask, jsonify, request
from flask_jwt_extended import JWTManager, create_access_token, jwt_required
```

Any Questions?

See you next time 😊

Dr. Ghadeer Mobasher

ghadeer.mobasher@srh-hochschulen.de

<https://www.linkedin.com/in/ghadeermobasher/>

LGS 6, 2. OG, Arc 213

